

A FFTRR Solver for the Poisson Equation over a Disk with Nonuniform Meshes

Charlie Pyle, Prabir Daripa

February 2026

Abstract

Mention ftrr class of algorithms One such algorithm was borges and daripa We Develop an efficient method for solving the Poisson equation on nonuniform grids both in the radial and azimuthal directions using the Inverse Nonuniform Fast Fourier Transform. This method is robust to nonuniform sampling grids, and extremely fast due to efficient vectorization within python. We release the solver on GitHub under ... We note the integration schemes may be modified as needed for desired or increased accuracy, and that this inverse method may be applied to other problems such as Helmholtz, etc....

Contents

1	Introduction	1
2	IMPORTANT	2
3	Problem Statement and Method Derivation	3
4	Integration Schemes	5
5	Uniform Algorithm Implementation	6
6	Nonuniform Azimuthal Meshes	8
6.1	Preliminaries	8
6.2	Azimuthal Mesh Algorithms	10
7	Complete NUFFTTRR Algorithm	14
7.1	Complete NUFFTTRR Algorithm	14
8	Experiments	16
8.1	Basic Example	16
8.2	FFTRR Method Comparisons	16
8.3	Different Problems, Meshes, Conditions, etc...	19
8.4	JCP Paper Problems	19
8.5	Comparison to other solvers	21
9	Conclusion	21
10	Possible Reference Papers and Notes	22

1 Introduction

Introduce problem, nonuniform issues

Background paragraph to briefly introduce Poisson Equation

One field of application for the Poisson equation is physics. It models a gravitational field as a function of mass density, [19, 2], and steady state heat conduction processes are modeled through the equation as well. In two dimensions, diffusion equations for steady state heat transfer are modeled using either Poisson equations or its homogeneous counterpart (Laplace Equations) [20]. Advective and Convective flows may be modeled in 3d over spheres using poissons equations, and have applications in meteorology, geophysics, and astrophysics [17, 9]. Within the context of Navier-Stokes Equations, Nonlinear Poisson Equations are used to model pressure fields [11, 18] and are essential in numerical solvers for Navier-Stokes problems.

Computer Science is another field in which poissons equations frequently appear. In surface reconstruction tasks, Poisson equations are used in Poisson surface reconstruction methods to reconstruct images from sample points [13]. This has many applications to the development of 2d and 3d scanners, Computer Assisted Design (CAD) systems, and computer vision. For example, Poisson's equations can be used in extracting shape properties of silhouettes within computer vision tasks [10]. In this scenario, many different Poisson equations must be solved and an efficient scheme to do so is of importance.

Regarding electronics, chemistry, and biology, we see many more applications. In [7], the authors apply a fast algorithm for the poisson equation (The very one this algorithm is based on) in order to study blood flow in catheterized arteries. Within electrochemistry, Poisson equations are essential in finding steady states and system response to electrostatic or chemical stimuli in membrane transport problems. [6]. In transistors, MOSFETs threshold voltages under different voltage dependency's and geometries are modeled with Poisson Equations. [14]. The Nonlinear Poisson equation is used in modeling semiconductors, for example in finding the spatial distribution of electrostatic potential in cylindrical semiconductors with metal contacts and dielectric layers [12]. Radial temperature distributions of gasses follow Poisson equations, and have an important role in determining the temperature and density distributions of neutron absorbing gases when exposed to a neutron flux. This is relevant to both nuclear reactors and the analysis of rocket systems [8].

paragraph on need for fast solver. robustness for nonuniform meshes

Literature Review and Related Works review of daripas paper and the other method from the other paper. Include other equation solvers and mention this method is extendable to it. mention inverse nufft solvers currently. mention nufft libraries. Mention possible parallelization or GPU speedups. Other solvers, why ours are better.

Many efficient solvers for the Poisson equation on the disk exists which use Fourier transforms along with finite difference schemes. For example, [15, 3] develop third and fourth order accurate schemes with complexity of $O(MN \log(N))$ for an $M \times N$ grid. These schemes however require a shift in points in order to deal with nonsingularity at the origin. Our methods using greens functions do not require this shift, and although higher order accurate finite difference schemes are easier to implement, our integration rules may be modified to any desired order to increase accuracy as well.

These solvers have been applied to other elliptic PDE's as well. For the Helmholtz equation, consider the respective finite difference [5] and FFTRR algorithm [1]. The latter of which considers the FFTRR method for both Helmholtz and Poisson equations on interior and exterior regions of circles, along with an annulus. Indeed our algorithm likewise could be extended to solve the helmholtz equation or these problems on the other domains. An extension of the fourth order accurate finite difference method for both the disk and sphere, with either dirichlet, neumann, or robin conditions, can be found in [16]

2 IMPORTANT

Dr. Daripa, I will need your help reviewing these articles to compare to our solver. most use finite differences, which i believe could be swapped with our quadrature. but relating to the transforms, complexity, accuracy, etc... they are different:

- https://heatherw3521.github.io/msc_thesis.pdf I believe they use chebshev-fourier transform to handle nonuniform cases. I believe they said their method is $O(mn)$, which would be better than ours

- <https://epubs.siam.org/doi/epdf/10.1137/16M1070207> this talks more about the chebyshev methods. they might have an optimal fast solver
- <https://academic.oup.com/ima/jna/article/40/3/1994/5621364?login=true> they don't do disk poisson, but they write "Since FFT-based fast Poisson solvers necessarily require uniform grids they cannot provide a practical optimal complexity spectral method". we could say that we now can address this problem
- <http://math.ncu.edu.tw/~wfh/papers/LHH20.pdf> is a recent paper with a fast solver

Also, right now I've noted that the NUDFT is faster even though not asymptotically than the nufft + cg method. I believe this holds for any feasible case ($M \approx 10000$). for the cg method, we can speed up a little bit using block conjugate gradient if each nonuniform azimuthal mesh is the same. if not, we have to do a separate cg for each radius. the nudft can be applied to either. so I say we mention the two cg methods but for experiments/practice use the less optimal nudft method.

Contributions Write our contributions, paper outline, what we do that's different, comparison, etc....

3 Problem Statement and Method Derivation

Solution to the Dirichlet Problem Consider the following equation with Dirichlet boundary conditions

Definition 1. Poisson Equation with Dirichlet Conditions

$$\begin{aligned}\Delta u &= f \quad \text{in } B, \\ u &= g \quad \text{on } \partial B,\end{aligned}$$

where $B = B(0, R) = \{x \in \mathbb{R}^2 : |x| < R\}$. (B is a disk)

We may simplify the equation by first splitting it into a homogeneous and nonhomogeneous solution. We split u into the sum of a particular solution v and the homogeneous correction w , so that v solves the unbounded problem and w imposes boundary corrections.

let v satisfy

$$\Delta v = f \quad \text{in } B,$$

and let w be the solution of the homogeneous problem

$$\begin{aligned}\Delta w &= 0 \quad \text{in } B, \\ w &= g - v \quad \text{on } \partial B.\end{aligned}$$

Then, the solution of the Dirichlet problem is given by

$$u = v + w.$$

The function v can be expressed directly in terms of the source function f using the Green's function for the Laplace operator. The Green's function $G(x, \eta)$ represents the influence at the observation point x caused by the unit source located at η .

In two dimensions, the free-space Green's function for the Laplacian is given by

$$G(x, \eta) = \frac{1}{2\pi} \log |x - \eta|.$$

By superposing the effects from all source points, we obtain

$$v(x) = \int_B f(\eta) G(x, \eta) d\eta, \quad x \in B,$$

A naive evaluation of $v(x)$ using quadrature runs into issues due to accuracy and would incur $O(N^4)$ complexity on the grid. We however make use of Fourier transformations in order to develop a fast and accurate algorithm.

Fourier decomposition Now, we work in polar coordinates and introduce the following solutions and operators in terms of fourier series. Since α is 2π -periodic, we expand u , f , and g in Fourier series in the Azimuthal variable:

$$\begin{aligned} u(r, \alpha) &= \sum_{n=-\infty}^{\infty} u_n(r) e^{in\alpha}, & u_n(r) &= \frac{1}{2\pi} \int_0^{2\pi} u(r, \alpha) e^{-in\alpha} d\alpha, \\ f(r, \alpha) &= \sum_{n=-\infty}^{\infty} f_n(r) e^{in\alpha}, & f_n(r) &= \frac{1}{2\pi} \int_0^{2\pi} f(r, \alpha) e^{-in\alpha} d\alpha, \\ g(\alpha) &= \sum_{n=-\infty}^{\infty} g_n e^{in\alpha}, & g_n &= \frac{1}{2\pi} \int_0^{2\pi} g(\alpha) e^{-in\alpha} d\alpha. \end{aligned}$$

We now introduce an important theorem from [4], which allows us to derive recursive relationships for evaluating one dimensional radial integrals in order to solve our problem efficiently.

Theorem 1. The n th Fourier coefficient $u_n(r)$ of the solution $u(r, \cdot)$ to the Dirichlet Poisson equation on the disk can be expressed as

$$u_n(r) = v_n(r) + \left(\frac{r}{R}\right)^{|n|} (g_n - v_n(R)), \quad 0 < r < R$$

where g_n are the Fourier coefficients of the boundary condition g and the following represents the Fourier coefficient of the particular solution:

$$v_n(r) = \int_0^r p_n(r, \rho) d\rho + \int_r^R q_n(r, \rho) d\rho.$$

and

$$\begin{aligned} p_n(r, \rho) &= \begin{cases} \rho \log r f_0(\rho), & n = 0, \\ -\frac{\rho}{2|n|} \left(\frac{\rho}{r}\right)^{|n|} f_n(\rho), & n \neq 0, \end{cases} \\ q_n(r, \rho) &= \begin{cases} \rho \log \rho f_0(\rho), & n = 0, \\ -\frac{\rho}{2|n|} \left(\frac{r}{\rho}\right)^{|n|} f_n(\rho), & n \neq 0. \end{cases} \end{aligned}$$

This formula converts the original 2D Green's function integral into 1D radial integrals for each Fourier mode.

Radial Recurrences We may use the following recursive relationships to efficiently compute the 1d integrals needed within our solutions for $v(x)$. We have the disk $B(0, R)$ with $N \times M$ lattice points, N equidistant points in the Azimuthal direction, and M distinct points in the radial direction. Note that the radial points may be nonuniform.

Let $0 = r_1 < r_2 < \dots < r_M = R$ be the radii defined on the discretizations grid. We first note that $v_n(0) = 0$ for $n \neq 0$. Then,

$$\begin{aligned} C_n^{i,j} &= \int_{r_i}^{r_j} \frac{\rho}{2n} \left(\frac{r_j}{\rho}\right)^n f_n(\rho) d\rho, \quad n < 0, \\ D_n^{i,j} &= - \int_{r_i}^{r_j} \frac{\rho}{2n} \left(\frac{r_i}{\rho}\right)^n f_n(\rho) d\rho, \quad n > 0. \end{aligned}$$

If for $r_j > r_i$, we define

$$\begin{aligned} v_n^-(r_1) &= 0, \quad n < 0, \\ v_n^-(r_j) &= \left(\frac{r_j}{r_i}\right)^n v_n^-(r_i) + C_n^{i,j}, \quad n < 0, \end{aligned}$$

$$v_n^+(r_M) = 0, \quad n > 0,$$

$$v_n^+(r_i) = \left(\frac{r_i}{r_j}\right)^n v_n^+(r_j) + D_n^{i,j}, \quad n > 0,$$

then for $i = 1, \dots, M$ we have

$$v_n(r_i) = \begin{cases} v_n^-(r_i) + \overline{v_{-n}^+(r_i)}, & n < 0, \\ v_n^+(r_i) + \overline{v_{-n}^-(r_i)}, & n > 0. \end{cases}$$

adding $n = 0$ to the definitions of C and D for initial terms:

$$C_0^{i,j} = \int_{r_i}^{r_j} \rho f_0(\rho) d\rho, \quad D_0^{i,j} = \int_{r_i}^{r_j} \rho \log \rho f_0(\rho) d\rho,$$

then given $l = 1, \dots, M$ we have

$$v_n(r_l) = \begin{cases} \log r_l \sum_{i=2}^l C_0^{i-1,i} + \sum_{i=l}^{M-1} D_0^{i,i+1}, & n = 0, \\ \sum_{i=2}^l \left(\frac{r_l}{r_i}\right)^n C_n^{i-1,i} + \sum_{i=l}^{M-1} \left(\frac{r_i}{r_l}\right)^n \overline{D_{-n}^{i,i+1}}, & n < 0, \\ \sum_{i=l}^{M-1} \left(\frac{r_l}{r_i}\right)^n D_n^{i,i+1} + \sum_{i=2}^l \left(\frac{r_i}{r_l}\right)^n \overline{C_{-n}^{i-1,i}}, & n > 0. \end{cases}$$

These formulas may be used to efficiently compute solutions to the Dirichlet Problem using the FFTRR algorithm.

Neumann Conditions The method for solving the Poisson Equation with Neumann Conditions is similar to the Dirichlet problem. First consider the Neumann problem:

Definition 2. Poisson Equation with Neumann Conditions

$$\begin{aligned} \Delta u &= f \quad \text{in } B, \\ \frac{\partial u}{\partial n} &= \psi \quad \text{on } \partial B. \end{aligned}$$

where $B = B(0, R) = \{x \in \mathbb{R}^2 : |x| < R\}$. (B is a disk)

Theorem 2. If $u(r, \alpha)$ is the solution of the Neumann problem, then the n th Fourier coefficient $u_n(r)$ of $u(r, \cdot)$ can be written as

$$u_0(r) = v_0(r) + \varphi_0, \quad n = 0$$

$$u_n(r) = v_n(r) + \left(\frac{r}{R}\right)^{|n|} \left(\frac{R}{|n|} \psi_n + v_n(R)\right), \quad n \neq 0$$

where ψ_n are the Fourier coefficients of ψ on ∂B , v_n are defined as before, and φ_0 is the parameter which sets the additive constant for the solution.

4 Integration Schemes

A straightforward way to approximate the radial integrals is to apply the trapezoidal rule, but this yields only second-order accuracy in the radial step size. One might try to improve the order by inserting auxiliary radii between the original grid points and then using higher-order quadrature formulas to compute $C_n^{i,i+1}$ and $D_n^{i,i+1}$ more accurately. This strategy, however, poses many issues, which are detailed within the original paper. To avoid these issues while still improving accuracy, some of the algorithms steps can be reformulated using the more general recurrences so that $C_n^{i,i+1}$ and $D_n^{i,i+1}$ are always formed from two neighboring radii

only. For example, when $n < 0$ and the radial mesh is uniform with spacing δr and radii $r_i = (i - 1) \delta r$, the trapezoidal rule applied between r_i and r_{i+1} gives

$$C_n^{i,i+1} = \frac{(\delta r)^2}{4n} \left[i \left(\frac{i}{i+1} \right)^{-n} f_n(r_i) + (i+1) f_n(r_{i+1}) \right],$$

which corresponds to integrating between r_i and r_{i+1} . An analogous expression holds for $D_n^{i,i+1}$.

If integrals of the form $C_n^{i-1,i+1}$ and $D_n^{i-1,i+1}$ are also evaluated, then three consecutive radii can be used in the quadrature, enabling Simpson's rule. For $n < 0$ and a uniform grid, one obtains

$$C_n^{i-1,i+1} = \frac{(\delta r)^2}{6n} \left[(i-1) \left(\frac{i-1}{i+1} \right)^{-n} f_n(r_{i-1}) + 4i \left(\frac{i}{i+1} \right)^{-n} f_n(r_i) + (i+1) f_n(r_{i+1}) \right],$$

which gives a higher-order approximation while still referencing only three neighboring circles. This will also lead to modified recurrences. For $n < 0$ (negative modes), we redefine:

$$v_n^-(r_1) = 0, \quad v_n^-(r_2) = C_n^{1,2},$$

and for $l = 3, \dots, M$,

$$v_n^-(r_l) = \left(\frac{r_l}{r_{l-2}} \right)^n v_n^-(r_{l-2}) + C_n^{l-2,l}.$$

Similarly, for $n > 0$ (positive modes), we define:

$$v_n^+(r_M) = 0, \quad v_n^+(r_{M-1}) = D_n^{M-1,M},$$

and for $l = M - 2, \dots, 1$,

$$v_n^+(r_l) = \left(\frac{r_l}{r_{l+2}} \right)^n v_n^+(r_{l+2}) + D_n^{l,l+2}.$$

These modified recurrences effectively apply Simpson-type integration over each triple of neighboring circles r_{i-1}, r_i, r_{i+1} , while keeping the cost per step comparable to that of the trapezoidal rule.

mention higher order schemes could be used, this could make accuracy the same or better as the paper using the finite difference, and doesn't pose the same issues with the singularity.

5 Uniform Algorithm Implementation

Sequential algorithm for the Dirichlet problem We now state the sequential FFTRR algorithm for solving the Dirichlet problem on a disk on a uniform grid in the Azimuthal variable α and a (possibly) nonuniform grid in the radial variable ρ .

Given:

- Radii $0 = \rho_1 < \rho_2 < \dots < \rho_M = R$.
- Azimuthal nodes $\alpha_k = 2\pi k/N$, $k = 1, \dots, N$.
- Data values $f(\rho_\ell, \alpha_k)$ for $\ell = 1, \dots, M$, $k = 1, \dots, N$.
- Boundary values $g(\alpha_k)$ on ∂B at $\rho = R$.

Goal: Compute the solution values

$$u(\rho_\ell, \alpha_k), \quad \ell = 1, \dots, M, \quad k = 1, \dots, N,$$

of the Dirichlet problem on the disk.

Step 1: Azimuthal Fourier transform. For each radius ρ_ℓ , use the fast Fourier Transform to compute the Fourier coefficients $f_n(\rho_\ell)$ of the right-hand side, and the boundary coefficients g_n of g :

$$f_n(\rho_\ell) = \frac{1}{N} \sum_{k=1}^N f(\rho_\ell, \alpha_k) e^{-in\alpha_k}, \quad g_n = \frac{1}{N} \sum_{k=1}^N g(\alpha_k) e^{-in\alpha_k},$$

for all modes $n \in [-N/2, N/2]$, $\ell = 1, \dots, M$.

Step 2: Radial integrals. For each radial interval $[\rho_i, \rho_{i+1}]$, $i = 1, \dots, M-1$, compute the one-dimensional integrals

$$C_n^{i,i+1}, \quad n \in [-N/2, 0], \quad D_n^{i,i+1}, \quad n \in [0, N/2],$$

according to the definitions of $C_n^{i,j}$ and $D_n^{i,j}$ in the radial recurrences section, using the chosen quadrature (trapezoidal or Simpson-type).

Step 3: Forward recurrence for negative modes. For each mode $n \in [-N/2, 0]$, compute $v_n^-(\rho_\ell)$ by sweeping outward in ρ :

1. Set

$$v_n^-(\rho_1) = 0, \quad n \in [-N/2, 0].$$

2. For $\ell = 2, \dots, M$, update

$$v_n^-(\rho_\ell) = \left(\frac{\rho_\ell}{\rho_{\ell-1}} \right)^n v_n^-(\rho_{\ell-1}) + C_n^{\ell-1,\ell}, \quad n \in [-N/2, 0].$$

Step 4: Backward recurrence for positive modes. For each mode $n \in [0, N/2]$, compute $v_n^+(\rho_\ell)$ by sweeping inward in ρ :

1. Set

$$v_n^+(\rho_M) = 0, \quad n \in [0, N/2].$$

2. For $\ell = M-1, \dots, 1$, update

$$v_n^+(\rho_\ell) = \left(\frac{\rho_\ell}{\rho_{\ell+1}} \right)^n v_n^+(\rho_{\ell+1}) + D_n^{\ell,\ell+1}, \quad n \in [0, N/2].$$

Step 5: Combine coefficients v_n^+ and v_n^- . For each radius index $\ell = 1, \dots, M$:

$$v_0(\rho_\ell) = \log \rho_\ell v_0^-(\rho_\ell) + v_0^+(\rho_\ell).$$

For the nonzero modes,

$$v_n(\rho_\ell) = \overline{v_{-n}(\rho_\ell)} = v_n^-(\rho_\ell) + \overline{v_{-n}^+(\rho_\ell)}, \quad n \in [-N/2, -1].$$

(For $n > 0$, the coefficients are obtained from the relation $v_n(\rho_\ell) = \overline{v_{-n}(\rho_\ell)}$.)

Step 6: Impose the Dirichlet boundary condition. For each radius ρ_ℓ and each mode $n \in [-N/2, N/2]$, define the full Fourier coefficients of the solution by

$$u_n(\rho_\ell) = v_n(\rho_\ell) + \left(\frac{\rho_\ell}{R} \right)^{|n|} (g_n - v_n(R)),$$

where $R = \rho_M$ and $v_n(R)$ is the value of the particular solution at the outer radius.

Step 7: Inverse Azimuthal Fourier transform. For each radius ρ_ℓ , reconstruct the physical-space solution on the Azimuthal grid:

$$u(\rho_\ell, \alpha_k) = \sum_{n=-N/2}^{N/2} u_n(\rho_\ell) e^{in\alpha_k}, \quad k = 1, \dots, N, \quad \ell = 1, \dots, M.$$

This completes the FFTRR-based sequential algorithm for the Dirichlet problem on the disk in polar coordinates (ρ, α) .

Efficient Implementation discuss numpy vectorization, and fft library.

Complexity Analysis

6 Nonuniform Azimuthal Meshes

For the case where we have a nonuniform azimuthal mesh

6.1 Preliminaries

Uniform FFT Preliminaries In the uniform algorithm, we rely on the FFT to move between Azimuthal samples and Fourier modes. Recall we use the forward FFT to convert Azimuthal samples to modes:

$$f_n(\rho_\ell) = \frac{1}{N} \sum_{k=1}^N f(\rho_\ell, \alpha_k) e^{-in\alpha_k}, \quad n \in [-N/2, N/2]$$

and the inverse fourier transform to compute fourier modes back to samples:

$$u(\rho_\ell, \alpha_k) = \sum_{n=-N/2}^{N/2} u_n(\rho_\ell) e^{in\alpha_k}, \quad k = 1, \dots, N$$

both with uniform nodes $a_k = \frac{2\pi k}{N}$.

It will be convenient to write these operations in matrix form. Consider an $N \times N$ Fourier matrix:

$$A_{k,n} = e^{in\alpha_k}, \quad k = 1, \dots, N, \quad n = -\frac{N}{2}, \dots, \frac{N}{2} - 1$$

for each radius ρ_ℓ , collecting Azimuthal samples into a vector $f^{(\ell)}$ and fourier coefficients into $\hat{f}^{(\ell)}$, we may define the following relations:

$$\begin{aligned} f^{(\ell)} &= A \hat{f}^{(\ell)} \\ \hat{f}^{(\ell)} &= A^{-1} f^{(\ell)} \end{aligned}$$

Where the first definition is the inverse transform (modes to samples) and the second is the forward transform (samples to modes) respectively. In the case of uniform nodes, the matrix A is unitary up to a factor of N, and thus its inverse A^{-1} is simply the conjugate transpose scaled $\frac{1}{N}A^*$. As a result, the forward and inverse transforms will recover the data when applied sequentially.

By stacking the radii column-wise into two $N \times M$ matrices F and $\hat{F}$ we obtain the relations

$$\begin{aligned} F^{(\ell)} &= A \hat{F}^{(\ell)} \\ \hat{F}^{(\ell)} &= A^{-1} F^{(\ell)} \end{aligned}$$

Naive evaluations of forward and inverse fast fourier transforms for the vectors both take $O(N^2)$ time. However, using the FFT, we can evaluate the FFTs in $O(N \log(N))$ time for a vector. In the matrix case, we do these operations along M columns, giving us a naive evaluation of $O(MN^2)$ and a fast version in $O(MN \log(N))$ time.

Nonuniform Case We now consider the case where the Azimuthal grid is nonuniform. For each radius ρ_ℓ , instead of equispaced nodes $\alpha_k = 2\pi k/N$ we have arbitrary angles

$$\alpha_k^{(\ell)} \in [0, 2\pi), \quad k = 1, \dots, N,$$

which may depend on the radius index ℓ . At fixed ρ_ℓ , the mapping of modes to samples generalizes to the nonuniform discrete Fourier transform, where we map a vector of Fourier coefficients $\hat{f}^{(\ell)}$ to its nonuniform Azimuthal samples $f^{(\ell)}$. The synthesis relation at each radius still has the same functional form,

$$f(\rho_\ell, \alpha_k^{(\ell)}) = \sum_{n=-N/2}^{N/2-1} \hat{f}_n^{(\ell)} e^{in\alpha_k^{(\ell)}}, \quad k = 1, \dots, N,$$

This evaluation also has a fast counterpart, which is frequently labeled as the type-2 nonuniform FFT (NUFFT-2) in the Azimuthal variable. It is also referred to as the forward NUFFT.

In matrix form we again write

$$f^{(\ell)} = A^{(\ell)} \hat{f}^{(\ell)},$$

where the entries of $A^{(\ell)}$ are now

$$A_{k,n}^{(\ell)} = e^{in\alpha_k^{(\ell)}}, \quad k = 1, \dots, N, \quad n = -\frac{N}{2}, \dots, \frac{N}{2} - 1,$$

with the $\alpha_k^{(\ell)}$ arbitrary. This is the nonuniform discrete Fourier transform (NUDFT) matrix. It has the same functional form as in the uniform case, but because the nodes are irregular, $A^{(\ell)}$ is no longer unitary. We also note that there is an adjoint operator $(A^{(\ell)})^*$ which corresponds to the NUFFT Type 1, converting nonuniform samples to uniform modes.

In the uniform case the key identity $A^{-1} = \frac{1}{N}A^*$ means that applying the adjoint (up to scaling) is the same as inverting A . This is precisely why a single FFT call recovers Fourier coefficients from samples.

For nonuniform nodes this identity breaks down: in general $(A^{(\ell)})^* \neq (A^{(\ell)})^{-1}$. Applying $(A^{(\ell)})^*$ to $f^{(\ell)}$ does not in general give $\hat{f}^{(\ell)}$. The type-1 NUFFT computes the adjoint, not the inverse. To recover Fourier coefficients from nonuniform samples (the analysis step) we must actually solve the following linear system for $\hat{f}^{(\ell)}$

$$A^{(\ell)} \hat{f}^{(\ell)} = f^{(\ell)}.$$

When the nodes $\alpha_k^{(\ell)}$ are distinct, $A^{(\ell)}$ is generically invertible (it is a square Vandermonde-like matrix). For numerical stability, the inversion is formulated as a least-squares problem:

$$\min_{\hat{f}^{(\ell)}} \|A^{(\ell)} \hat{f}^{(\ell)} - f^{(\ell)}\|_2,$$

whose solution satisfies the normal equations

$$(A^{(\ell)})^* A^{(\ell)} \hat{f}^{(\ell)} = (A^{(\ell)})^* f^{(\ell)}.$$

In operator form, the analysis step in the nonuniform case is therefore

$$\hat{f}^{(\ell)} = ((A^{(\ell)})^* A^{(\ell)})^{-1} (A^{(\ell)})^* f^{(\ell)},$$

replacing the simple $\hat{f}^{(\ell)} = A^{-1} f^{(\ell)}$ of the uniform case. The synthesis step remains unchanged:

$$f^{(\ell)} = A^{(\ell)} \hat{f}^{(\ell)}.$$

Stacking all radii columnwise into $N \times M$ matrices

$$F = [f^{(1)} \dots f^{(M)}], \quad \hat{F} = [\hat{f}^{(1)} \dots \hat{f}^{(M)}],$$

we may write the nonuniform Azimuthal transforms compactly as

$$F = A^{(\ell)} \hat{F}, \quad \hat{F} = ((A^{(\ell)})^* A^{(\ell)})^{-1} (A^{(\ell)})^* F,$$

6.2 Azimuthal Mesh Algorithms

Let us now review the four azimuthal mesh algorithms. Suppose we are given a function $f(r, \alpha)$ and the boundary solution function $g(\alpha)$. Suppose for samples there are M radial points and N azimuthal points. We collect these samples in an $N \times M$ data matrix F and length N vector g defined as follows:

$$\begin{aligned} F_{k,\ell} &= f(\rho_\ell, \alpha_k), & k &= 1, \dots, N, \ell = 1, \dots, M, \\ g_k &= g(\alpha_k), & k &= 1, \dots, N, \end{aligned}$$

For each fixed radius ρ_ℓ , we regard the column vector as an N -point periodic signal in the Azimuthal variable α .

$$F_{\cdot,\ell} = (f(\rho_\ell, \alpha_0), \dots, f(\rho_\ell, \alpha_{N-1}))^T$$

The azimuthal portion of the FFTRR algorithm consists of an analysis step, in which the columns of F and the vector g are mapped to their Azimuthal Fourier coefficients, and a synthesis step, in which the Fourier coefficients of the solution are transformed back to physical-space values on the azimuthal grid.

Uniform Mesh In the uniform case the azimuthal nodes are

$$\alpha_k = \frac{2\pi k}{N}, \quad k = 0, \dots, N-1,$$

For the Analysis step, we then apply a one-dimensional FFT in α to each column of F and to the boundary data vector g , giving us the Azimuthal Fourier coefficients

$$\begin{aligned} \hat{f}_n(\rho_\ell) &= \frac{1}{N} \sum_{k=0}^{N-1} f(\rho_\ell, \alpha_k) e^{-in\alpha_k}, & n &= -\frac{N}{2}, \dots, \frac{N}{2}, \ell = 1, \dots, M, \\ \hat{g}_n &= \frac{1}{N} \sum_{k=0}^{N-1} g(\alpha_k) e^{-in\alpha_k}, & n &= -\frac{N}{2}, \dots, \frac{N}{2}. \end{aligned}$$

Each FFT of length N costs $O(N \log N)$ operations, so computing all $\hat{f}_n(\rho_\ell)$ for $\ell = 1, \dots, M$ and $\hat{g}_n$ costs $O(MN \log N)$ in total.

For the synthesis step, the radial FFTRR recurrences are first applied mode-by-mode to the analyzed data $\hat{f}_n(\rho_\ell)$ and $\hat{g}_n$, producing the Fourier coefficients $\hat{u}_n(\rho_\ell)$ of the solution. We then apply a one-dimensional inverse FFT in α to each column of the coefficient array, reconstructing

$$u(\rho_\ell, \alpha_k) = \sum_{n=-N/2}^{N/2} \hat{u}_n(\rho_\ell) e^{in\alpha_k}, \quad k = 0, \dots, N-1, \ell = 1, \dots, M.$$

Again, this requires one inverse FFT of length N per radius, so the synthesis step also costs $O(MN \log N)$.

Our total azimuthal runtime in the uniform case is therefore $O(MN \log N)$, coming from one forward and one inverse FFT of length N applied to all M radii. In practice these transforms are implemented as vectorized, batched FFT calls along the Azimuthal dimension, so that all columns are processed simultaneously in memory.

NUDFT on Partially Non Uniform Mesh In the partially nonuniform case the azimuthal grid is nonuniform but shared across all radii:

$$\alpha_k \in [0, 2\pi), \quad k = 0, \dots, N-1,$$

with the same set of angles $\{\alpha_k\}$ used for every ρ_ℓ . At fixed radius ρ_ℓ the column $F_{\cdot,\ell}$ is thus sampled on a common nonuniform Azimuthal mesh.

Analysis Step. For each radius ρ_ℓ we seek Fourier coefficients $\hat{f}_n(\rho_\ell)$ such that

$$f(\rho_\ell, \alpha_k) \approx \sum_{n=-N/2}^{N/2-1} \hat{f}_n(\rho_\ell) e^{in\alpha_k}, \quad k = 0, \dots, N-1.$$

Collecting these into vectors, this is the nonuniform discrete Fourier transform (NUDFT) relation

$$F_{\cdot,\ell} \approx A \hat{f}^{(\ell)}, \quad A_{k,n} = e^{in\alpha_k},$$

where $\hat{f}^{(\ell)}$ stores the modes $\hat{f}_n(\rho_\ell)$. Since the nodes $\{\alpha_k\}$ are nonuniform, the analysis step is posed as the following least-squares problem and normal equation

$$\begin{aligned} \min_{\hat{f}^{(\ell)}} \|A \hat{f}^{(\ell)} - F_{\cdot,\ell}\|_2, \\ (A^* A) \hat{f}^{(\ell)} = A^* F_{\cdot,\ell}. \end{aligned}$$

Because the matrix A is the same for all radii, we form $A^* A$ and its factorization once, then solve these normal equations for all $\ell = 1, \dots, M$. Applying the same procedure to the boundary data g gives the coefficients $\hat{g}_n$. The normal-equation matrix $M := A^* A$ is of size $N \times N$. Forming M and computing its (e.g. Cholesky or LU) factorization costs $O(N^3)$. For each radius ρ_ℓ , assembling the right-hand side $A^* F_{\cdot,\ell}$ and solving $M \hat{f}^{(\ell)} = A^* F_{\cdot,\ell}$ requires $O(N^2)$ operations. Thus, across all M radii, the analysis step for f and the analogous step for g together incur a total cost of order $O(N^3 + MN^2)$.

Synthesis Step. After the radial FFTRR recurrences are applied to the analyzed data $\hat{f}_n(\rho_\ell)$ and $\hat{g}_n$, we obtain solution coefficients $\hat{u}_n(\rho_\ell)$. To reconstruct the solution on the nonuniform Azimuthal grid we evaluate

$$u(\rho_\ell, \alpha_k) = \sum_{n=-N/2}^{N/2} \hat{u}_n(\rho_\ell) e^{in\alpha_k}, \quad k = 0, \dots, N-1, \quad \ell = 1, \dots, M,$$

using a type-2 nonuniform FFT (NUFFT-2) in the Azimuthal variable. In our implementation this is realized by a single FINUFFT call with the shared node set $\{\alpha_k\}_{k=0}^{N-1}$ and M simultaneous transforms, so that all radii are synthesized in one vectorized operation on the (N, M) coefficient array. For a fixed tolerance ε , the cost of a 1D NUFFT-2 with N modes and N nonuniform targets is $O(N \log(\varepsilon^{-1}) + N \log N)$, and multiplying by the M batched transforms gives an overall synthesis complexity of $O(MN \log(\varepsilon^{-1}) + MN \log N)$.

Total Complexity. Combining the analysis and synthesis stages, the azimuthal cost of the partially nonuniform NUDFT method is

$$O(N^3 + MN^2) + O(MN \log(\varepsilon^{-1}) + MN \log N).$$

For fixed tolerance ε , the dominant terms as N and M grow are N^3 (for forming and factoring $A^* A$) and MN^2 (for solving the normal equations at each radius). Thus, asymptotically, the overall azimuthal complexity of the fixed NUDFT variant is $O(N^3 + MN^2)$.

NUDFT on Completely Nonuniform Mesh In the fully nonuniform case each radius has its own azimuthal grid $\{\alpha_k^{(\ell)}\}_{k=0}^{N-1}$, so the Azimuthal nodes vary with ℓ . Thus, for fixed ρ_ℓ the column $F_{\cdot,\ell}$ is sampled on a radius-dependent nonuniform Azimuthal mesh.

Analysis Step. For each radius ρ_ℓ we seek Fourier coefficients $\hat{f}_n(\rho_\ell)$ such that

$$f(\rho_\ell, \alpha_k^{(\ell)}) \approx \sum_{n=-N/2}^{N/2-1} \hat{f}_n(\rho_\ell) e^{in\alpha_k^{(\ell)}}, \quad k = 0, \dots, N-1.$$

Collecting these into vectors, this is the NUDFT relation

$$F_{\cdot,\ell} \approx A^{(\ell)} \hat{f}^{(\ell)}, \quad A_{k,n}^{(\ell)} = e^{in\alpha_k^{(\ell)}},$$

where $\hat{f}^{(\ell)}$ stores the modes $\hat{f}_n(\rho_\ell)$. Since the nodes $\{\alpha_k^{(\ell)}\}$ are nonuniform and now depend on ℓ , the analysis step is posed for each radius as the following least-squares problem and normal equation

$$\begin{aligned} \min_{\hat{f}^{(\ell)}} \|A^{(\ell)} \hat{f}^{(\ell)} - F_{\cdot,\ell}\|_2, \\ (A^{(\ell)})^* A^{(\ell)} \hat{f}^{(\ell)} = (A^{(\ell)})^* F_{\cdot,\ell}. \end{aligned}$$

Unlike the partially nonuniform case, the matrices $A^{(\ell)}$ (and hence $(A^{(\ell)})^* A^{(\ell)}$) differ with ℓ , so no single factorization can be reused across radii. For each ℓ , the normal-equation matrix $M^{(\ell)} := (A^{(\ell)})^* A^{(\ell)}$ is of size $N \times N$. Forming $M^{(\ell)}$ and computing its (e.g. Cholesky or LU) factorization costs $O(N^3)$. Assembling the right-hand side $(A^{(\ell)})^* F_{\cdot,\ell}$ and solving $M^{(\ell)} \hat{f}^{(\ell)} = (A^{(\ell)})^* F_{\cdot,\ell}$ requires $O(N^2)$ operations. Thus, across all M radii, the analysis step for f and the analogous step for g together incur a total cost of order $O(MN^3)$.

Synthesis Step. After the radial FFTRR recurrences are applied to the analyzed data $\hat{f}_n(\rho_\ell)$ and $\hat{g}_n$, we obtain solution coefficients $\hat{u}_n(\rho_\ell)$. To reconstruct the solution on each nonuniform Azimuthal grid we evaluate

$$u(\rho_\ell, \alpha_k^{(\ell)}) = \sum_{n=-N/2}^{N/2} \hat{u}_n(\rho_\ell) e^{in\alpha_k^{(\ell)}}, \quad k = 0, \dots, N-1, \quad \ell = 1, \dots, M,$$

using a type-2 nonuniform FFT (NUFFT-2) in the Azimuthal variable. In our implementation this is done by a separate FINUFFT call for each radius ℓ , with its own node set $\{\alpha_k^{(\ell)}\}_{k=0}^{N-1}$ and a single transform. For a fixed tolerance ε , the cost of a 1D NUFFT-2 with N modes and N nonuniform targets is $O(N \log(\varepsilon^{-1}) + N \log N)$, so summing over all M radii gives an overall synthesis complexity of

$$O(MN \log(\varepsilon^{-1}) + MN \log N).$$

Total Complexity. Combining the analysis and synthesis stages, the azimuthal cost of the fully nonuniform NUDFT method is

$$O(MN^3) + O(MN \log(\varepsilon^{-1}) + MN \log N).$$

For fixed tolerance ε , the dominant term as N and M grow is MN^3 , coming from the per-radius NUDFT normal-equation solves, since $N^3 \gg N \log N$ for large N . Thus, asymptotically, the overall azimuthal complexity of the fully nonuniform NUDFT variant is

$$\text{cost}_\theta^{\text{NF-NU NUDFT}} = O(MN^3),$$

NUFFT+CG on Partially Nonuniform Mesh In this variant the azimuthal grid is again nonuniform but shared across all radii, $\alpha_k \in [0, 2\pi)$, $k = 0, \dots, N-1$, with the same node set $\{\alpha_k\}$ at every ρ_ℓ . Rather than forming the NUDFT matrix A explicitly, both the analysis and synthesis steps are implemented entirely through NUFFT calls.

Analysis Step. As in the NUDFT case, for each radius ρ_ℓ we seek Fourier coefficients $\hat{f}_n(\rho_\ell)$ satisfying

$$F_{\cdot,\ell} \approx A \hat{f}^{(\ell)}, \quad A_{k,n} = e^{in\alpha_k},$$

and pose the analysis as the least-squares problem with normal equations

$$\begin{aligned} \min_{\hat{f}^{(\ell)}} \|A \hat{f}^{(\ell)} - F_{\cdot,\ell}\|_2, \\ (A^* A) \hat{f}^{(\ell)} = A^* F_{\cdot,\ell}. \end{aligned}$$

Instead of forming and factoring $A^* A$ explicitly, we solve these normal equations iteratively using block conjugate gradient (block CG), applying the operator $T = A^* A$ as a matrix-free operation. Each application of T to a block of vectors is computed as:

1. a type-2 NUFFT: modes $\rightarrow$ nonuniform samples (applies A),
2. a type-1 NUFFT: nonuniform samples $\rightarrow$ modes (applies A^*).

Since the mesh is shared across all M radii, we create a single pair of FINUFFT plans (one type-1, one type-2) with `n.trans` = M , and at each CG iteration apply both plans simultaneously to the entire (N, M) block of iterates. The right-hand side $A^* F$ is formed once via a single batched type-1 NUFFT call. The same procedure applied to the boundary vector g yields $\hat{g}_n$.

For fixed tolerance ε , each NUFFT call (type-1 or type-2) with N modes, N nonuniform targets, and M simultaneous transforms costs

$$O(MN \log(\varepsilon^{-1}) + MN \log N),$$

so each block CG iteration costs the same. Letting K_{CG} denote the number of iterations to convergence, the total analysis cost is

$$O(K_{\text{CG}} MN \log(\varepsilon^{-1}) + K_{\text{CG}} MN \log N).$$

Synthesis Step. The synthesis step is identical to the Fix-NU NUDFT case: a single batched type-2 NUFFT call with the shared node set $\{\alpha_k\}$ and M simultaneous transforms reconstructs the solution values

$$u(\rho_\ell, \alpha_k) = \sum_{n=-N/2}^{N/2} \hat{u}_n(\rho_\ell) e^{in\alpha_k}, \quad k = 0, \dots, N-1, \quad \ell = 1, \dots, M,$$

on the full (N, M) array in one vectorized operation, at cost

$$O(MN \log(\varepsilon^{-1}) + MN \log N).$$

Total Complexity. Combining analysis and synthesis, the azimuthal cost of the Fix-NU NUFFT+CG method is

$$O(K_{\text{CG}} MN \log(\varepsilon^{-1}) + K_{\text{CG}} MN \log N) + O(MN \log(\varepsilon^{-1}) + MN \log N).$$

Since $K_{\text{CG}} \geq 1$, the analysis dominates, and the overall azimuthal complexity is

$$\text{cost}_\theta^{\text{Fix-NU NUFFT}} = O(K_{\text{CG}} MN \log N),$$

for fixed ε . In practice K_{CG} is a small constant (at most 50 in our implementation) that depends on the condition number of A^*A and the prescribed tolerance, but does not grow with N or M for well-separated nonuniform grids.

NUFFT+CG on Completely Nonuniform Mesh In this variant each radius has its own azimuthal grid $\{\alpha_k^{(\ell)}\}_{k=0}^{N-1}$, so the Azimuthal nodes vary with ℓ . As in the NF-NU NUDFT case, the NUDFT matrices $A^{(\ell)}$ differ at every radius, but here we again avoid forming them explicitly and instead rely entirely on NUFFT calls.

Analysis Step. For each radius ρ_ℓ we seek Fourier coefficients $\hat{f}_n(\rho_\ell)$ satisfying

$$F_{\cdot, \ell} \approx A^{(\ell)} \hat{f}^{(\ell)}, \quad A_{k, n}^{(\ell)} = e^{in\alpha_k^{(\ell)}},$$

via the normal equations

$$\min_{\hat{f}^{(\ell)}} \|A^{(\ell)} \hat{f}^{(\ell)} - F_{\cdot, \ell}\|_2, \\ (A^{(\ell)})^* A^{(\ell)} \hat{f}^{(\ell)} = (A^{(\ell)})^* F_{\cdot, \ell}.$$

Because the meshes differ with ℓ , there is no shared FINUFFT plan. Instead, for each radius ρ_ℓ we create a dedicated pair of FINUFFT plans (one type-1, one type-2) with the nodes $\{\alpha_k^{(\ell)}\}$ and `n.trans`= 1, and solve the normal equations by CG applied to the operator

$$T^{(\ell)} x := (A^{(\ell)})^* A^{(\ell)} x,$$

implemented as a type-2 NUFFT followed by a type-1 NUFFT at radius ρ_ℓ . The right-hand side $(A^{(\ell)})^* F_{\cdot, \ell}$ is formed once via a type-1 NUFFT call. This loop over radii is unavoidable since the node sets differ at each ℓ . The same procedure applied to the boundary samples $g_k = g(\alpha_k^{(M)})$ yields $\hat{g}_n$.

For fixed tolerance ε , each NUFFT call (type-1 or type-2) at radius ρ_ℓ with N modes and N nonuniform targets costs

$$O(N \log(\varepsilon^{-1}) + N \log N).$$

Each CG iteration at radius ρ_ℓ applies $T^{(\ell)}$ once, i.e. two NUFFT calls. Letting K_ℓ denote the number of CG iterations at radius ℓ , the analysis cost at that radius is

$$O(K_\ell N \log(\varepsilon^{-1}) + K_\ell N \log N),$$

and summing over all M radii gives

$$O\left(\sum_{\ell=1}^M K_{\ell} N \log(\varepsilon^{-1}) + \sum_{\ell=1}^M K_{\ell} N \log N\right).$$

Synthesis Step. Since the node sets differ with ℓ , the synthesis cannot be performed in a single batched NUFFT call. Instead, for each radius ρ_{ℓ} we evaluate

$$u(\rho_{\ell}, \alpha_k^{(\ell)}) = \sum_{n=-N/2}^{N/2} \hat{u}_n(\rho_{\ell}) e^{in\alpha_k^{(\ell)}}, \quad k = 0, \dots, N-1,$$

via a dedicated type-2 FINUFFT call with nodes $\{\alpha_k^{(\ell)}\}$ and `n.trans`= 1, looping over $\ell = 1, \dots, M$. Each call costs $O(N \log(\varepsilon^{-1}) + N \log N)$, so the total synthesis cost is

$$O(MN \log(\varepsilon^{-1}) + MN \log N).$$

Total Complexity. Combining analysis and synthesis, the azimuthal cost of the NF-NU NUFFT+CG method is

$$O\left(\sum_{\ell=1}^M K_{\ell} N \log(\varepsilon^{-1}) + \sum_{\ell=1}^M K_{\ell} N \log N\right) + O(MN \log(\varepsilon^{-1}) + MN \log N).$$

Since $K_{\ell} \geq 1$, the analysis dominates, and the overall azimuthal complexity is

$$\text{cost}_{\theta}^{\text{NF-NU NUFFT}} = O\left(\sum_{\ell=1}^M K_{\ell} N \log N\right),$$

for fixed ε . In practice the CG iteration counts K_{ℓ} are bounded by a fixed cap (at most 50 in our implementation) and do not grow with N or M for well-separated nonuniform grids, giving an effective azimuthal cost of $O(MN \log N)$ with a somewhat larger constant than the shared-mesh NUFFT+CG variant.

Table 1: Asymptotic azimuthal runtime for the five Azimuthal methods (with N azimuthal points, M radii, and fixed tolerance ε).

Method	Mesh type	Total azimuthal cost
Uniform FFT	uniform	$O(MN \log N)$
Fix-NU NUDFT	shared nonuniform	$O(N^3 + MN^2)$
Fix-NU NUFFT+CG	shared nonuniform	$O(K_{\text{CG}} MN \log N)$
NF-NU NUDFT	fully nonuniform	$O(MN^3)$
NF-NU NUFFT+CG	fully nonuniform	$O\left(\left(\sum_{\ell=1}^M K_{\ell}\right) N \log N\right)$

7 Complete NUFFTRR Algorithm

7.1 Complete NUFFTRR Algorithm

We now state the sequential NUFFTRR algorithm for solving the Dirichlet problem on a disk on a (possibly) nonuniform grid in both the Azimuthal variable α and the radial variable ρ .

Given:

- Radii $0 = \rho_1 < \rho_2 < \dots < \rho_M = R$.
- Azimuthal nodes $\alpha_k^{(\ell)} \in [0, 2\pi)$, $k = 0, \dots, N-1$, $\ell = 1, \dots, M$ (all equal to α_k in the partially nonuniform case; equispaced $\alpha_k = 2\pi k/N$ in the uniform case).

- Data values $f(\rho_\ell, \alpha_k^{(\ell)})$ for $\ell = 1, \dots, M$, $k = 0, \dots, N-1$.
- Boundary values $g(\alpha_k^{(M)})$ on ∂B at $\rho = R$.

Goal: Compute the solution values

$$u(\rho_\ell, \alpha_k^{(\ell)}), \quad \ell = 1, \dots, M, \quad k = 0, \dots, N-1,$$

of the Dirichlet problem on the disk.

Step 1: Azimuthal analysis (forward transform). Map the data matrix F and boundary vector g to their Azimuthal Fourier coefficients. The procedure depends on the mesh type:

- *Uniform mesh.* Apply a batched FFT along the Azimuthal dimension of F and to g :

$$\hat{f}_n(\rho_\ell) = \frac{1}{N} \sum_{k=0}^{N-1} f(\rho_\ell, \alpha_k) e^{-in\alpha_k}, \quad \hat{g}_n = \frac{1}{N} \sum_{k=0}^{N-1} g(\alpha_k) e^{-in\alpha_k},$$

for $n \in [-N/2, N/2]$, $\ell = 1, \dots, M$. Cost: $O(MN \log N)$.

- *Partially nonuniform mesh (shared nodes).* Form and factor $M := A^*A$ once, then solve $M \hat{f}^{(\ell)} = A^* F_{\cdot, \ell}$ for all $\ell = 1, \dots, M$ simultaneously, and solve $M \hat{g} = A^* g$ for the boundary coefficients. Cost: $O(N^3 + MN^2)$. Alternatively, apply block CG with batched NUFFT matvecs. Cost: $O(K_{\text{CG}} MN \log N)$.
- *Fully nonuniform mesh (per-radius nodes).* For each $\ell = 1, \dots, M$, form and factor $M^{(\ell)} := (A^{(\ell)})^* A^{(\ell)}$ and solve $M^{(\ell)} \hat{f}^{(\ell)} = (A^{(\ell)})^* F_{\cdot, \ell}$. Cost: $O(MN^3)$. Alternatively, apply per-radius CG with NUFFT matvecs. Cost: $O\left(\sum_{\ell=1}^M K_\ell N \log N\right)$.

This step produces the Fourier coefficient arrays $\hat{F} \in \mathbb{C}^{(N+1) \times M}$ and vector $\hat{g} \in \mathbb{C}^{N+1}$.

Step 2: Radial integrals. For each radial interval $[\rho_i, \rho_{i+1}]$, $i = 1, \dots, M-1$, compute the one-dimensional integrals

$$C_n^{i,i+1}, \quad n \in [-N/2, 0], \quad D_n^{i,i+1}, \quad n \in [0, N/2],$$

using the chosen quadrature rule (trapezoidal or Simpson-type). This step is identical for all mesh variants, since the Fourier coefficients $\hat{f}_n(\rho_\ell)$ are now defined on the common radial grid regardless of the Azimuthal mesh.

Step 3: Forward recurrence for negative modes. For each mode $n \in [-N/2, 0]$, sweep outward in ρ :

1. Set $v_n^-(\rho_1) = 0$.
2. For $\ell = 2, \dots, M$, update

$$v_n^-(\rho_\ell) = \left(\frac{\rho_\ell}{\rho_{\ell-1}} \right)^n v_n^-(\rho_{\ell-1}) + C_n^{\ell-1, \ell}.$$

Step 4: Backward recurrence for positive modes. For each mode $n \in [0, N/2]$, sweep inward in ρ :

1. Set $v_n^+(\rho_M) = 0$.
2. For $\ell = M-1, \dots, 1$, update

$$v_n^+(\rho_\ell) = \left(\frac{\rho_\ell}{\rho_{\ell+1}} \right)^n v_n^+(\rho_{\ell+1}) + D_n^{\ell, \ell+1}.$$

Step 5: Combine coefficients v_n^+ and v_n^- . For each radius index $\ell = 1, \dots, M$:

$$v_0(\rho_\ell) = \log \rho_\ell v_0^-(\rho_\ell) + v_0^+(\rho_\ell),$$

and for nonzero modes

$$v_n(\rho_\ell) = v_n^-(\rho_\ell) + \overline{v_{-n}^+(\rho_\ell)}, \quad n \in [-N/2, -1].$$

For $n > 0$, use $v_n(\rho_\ell) = \overline{v_{-n}(\rho_\ell)}$.

Step 6: Impose the Dirichlet boundary condition. For each radius ρ_ℓ and mode $n \in [-N/2, N/2]$,

$$\hat{u}_n(\rho_\ell) = v_n(\rho_\ell) + \left(\frac{\rho_\ell}{R}\right)^{|n|} (\hat{g}_n - v_n(R)).$$

Step 7: Azimuthal synthesis (inverse transform). Reconstruct physical-space values from the solution coefficient array $\hat{U} \in \mathbb{C}^{(N+1) \times M}$. The procedure again depends on the mesh type:

- *Uniform mesh.* Apply a batched inverse FFT along the Azimuthal dimension:

$$u(\rho_\ell, \alpha_k) = \sum_{n=-N/2}^{N/2} \hat{u}_n(\rho_\ell) e^{in\alpha_k}, \quad k = 0, \dots, N-1, \ell = 1, \dots, M.$$

Cost: $O(MN \log N)$.

- *Partially nonuniform mesh (shared nodes).* Apply a single batched NUFFT-2 call with the shared node set $\{\alpha_k\}_{k=0}^{N-1}$ and M simultaneous transforms, reconstructing all (N, M) values in one vectorized operation. Cost: $O(MN \log N)$.
- *Fully nonuniform mesh (per-radius nodes).* For each radius $\ell = 1, \dots, M$, apply a dedicated NUFFT-2 call with nodes $\{\alpha_k^{(\ell)}\}$ to recover $u(\rho_\ell, \alpha_k^{(\ell)})$, $k = 0, \dots, N-1$. Cost: $O(MN \log N)$.

This completes the NUFFTRR sequential algorithm for the Dirichlet problem on the disk in polar coordinates (ρ, α) .

8 Experiments

8.1 Basic Example

choose one problem, include error graph, results, etc...

8.2 FFTRR Method Comparisons

IMPORTANT: these are placeholders, likely need to do multiple runs and more testing on a better computer

Solver Conditions For all tests in this subsection we solve the Dirichlet problem

$$\Delta u = f \quad \text{in } B(0, R), \quad u = g \quad \text{on } \partial B(0, R),$$

with the solution

$$u(x, y) = x^2 + 3y^3, \quad f(x, y) = 2 + 18y, \quad g(x, y) = u(x, y),$$

on a disk of radius $R = 1$. we keep the radial mesh uniform and use the trapezoidal rule.

We consider five Azimuthal treatments:

1. *uniform_fft*: uniform angles with standard FFT in θ ;
2. *fixed_nonunif_nudft*: fixed nonuniform angles, shared across radii, with an exact NUDFT inversion;

3. *fixed_nonunif_nufft_cg*: fixed nonuniform angles with a NUFFT+block-CG inversion;
4. *nonfixed_nonunif_nudft*: fully nonuniform angles (different mesh per radius) with a per-radius NUDFT;
5. *nonfixed_nonunif_nufft_cg*: fully nonuniform angles with per-radius NUFFT+CG.

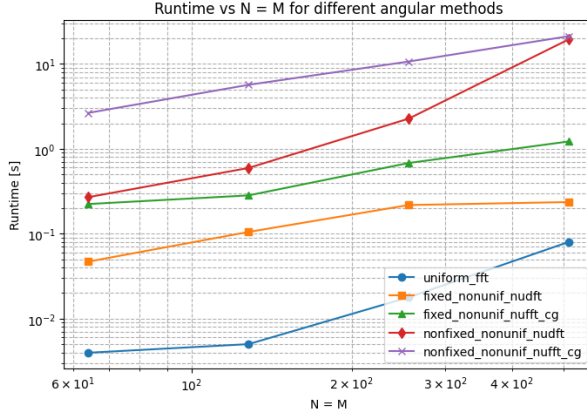
For all nonuniform cases we truncate the Fourier series symmetrically and use CMCL ordering of modes. The NUFFT calls are implemented via FINUFFT, and the NUDFT path uses dense normal-equation solves. All CG runs use a modest fixed tolerance 10^{-8} and at most 50 iterations.[?, ?, ?, ?]

Table 2: $N = M$: relative errors vs. N .

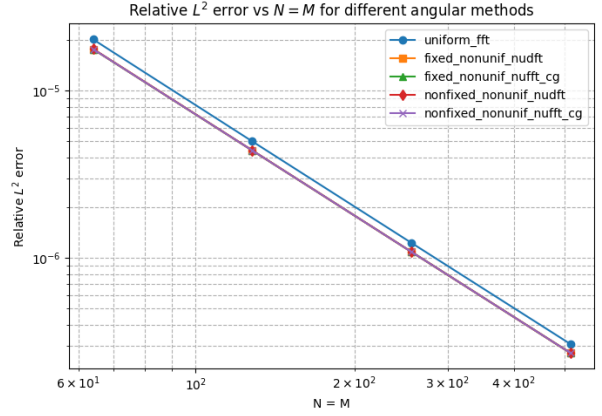
N	FFT		Fix-NU NUDFT		Fix-NU NUFFT		NF-NU NUDFT		NF-NU NUFFT	
	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2
64	9.978e-05	2.017e-05	9.998e-05	1.765e-05	1.001e-04	1.765e-05	1.001e-04	1.766e-05	1.001e-04	1.758e-05
128	2.697e-05	4.974e-06	2.699e-05	4.387e-06	2.698e-05	4.386e-06	2.698e-05	4.381e-06	2.699e-05	4.383e-06
256	7.285e-06	1.234e-06	7.286e-06	1.091e-06	7.286e-06	1.092e-06	7.286e-06	1.091e-06	7.286e-06	1.092e-06
512	1.962e-06	3.075e-07	1.962e-06	2.720e-07	1.962e-06	2.724e-07	1.962e-06	2.720e-07	1.962e-06	2.717e-07

Table 3: $N = M$: runtimes [s] vs. N .

N	FFT	Fix-NU NUDFT	Fix-NU NUFFT	NF-NU NUDFT	NF-NU NUFFT
64	3.960e-03	4.690e-02	2.236e-01	2.692e-01	2.648e+00
128	5.001e-03	1.049e-01	2.827e-01	5.950e-01	5.673e+00
256	1.784e-02	2.177e-01	6.812e-01	2.271e+00	1.068e+01
512	8.024e-02	2.360e-01	1.223e+00	1.956e+01	2.126e+01



(a) Runtime vs. $N = M$.



(b) Relative L^2 error vs. $N = M$.

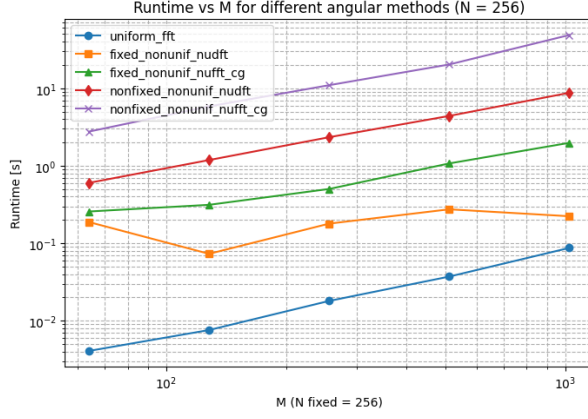
Figure 1: Timing and accuracy of the five Azimuthal methods when $N = M$.

$N = M$

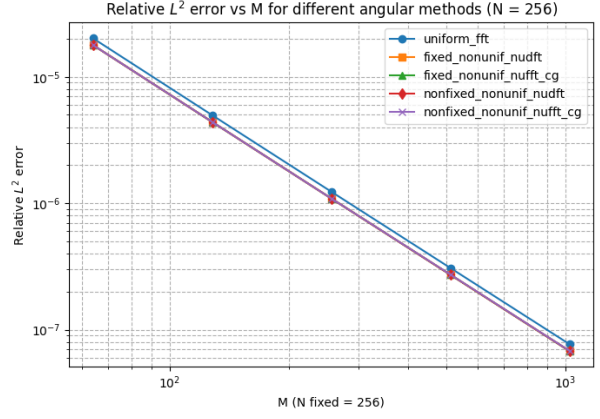
Table 4: Fixed N (here $N = 256$): relative errors vs. M .

M	FFT		Fix-NU NUDFT		Fix-NU NUFFT		NF-NU NUDFT		NF-NU NUFFT	
	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2
64	9.978e-05	2.017e-05	9.979e-05	1.783e-05	9.979e-05	1.783e-05	9.979e-05	1.782e-05	9.979e-05	1.783e-05
128	2.697e-05	4.974e-06	2.697e-05	4.398e-06	2.697e-05	4.396e-06	2.697e-05	4.397e-06	2.697e-05	4.399e-06
256	7.285e-06	1.234e-06	7.286e-06	1.091e-06	7.286e-06	1.092e-06	7.286e-06	1.091e-06	7.286e-06	1.092e-06
512	1.962e-06	3.075e-07	1.962e-06	2.719e-07	1.962e-06	2.719e-07	1.962e-06	2.719e-07	1.963e-06	2.728e-07
1024	5.264e-07	7.672e-08	5.264e-07	6.782e-08	5.264e-07	6.771e-08	5.265e-07	6.777e-08	5.264e-07	6.802e-08

Fixed N

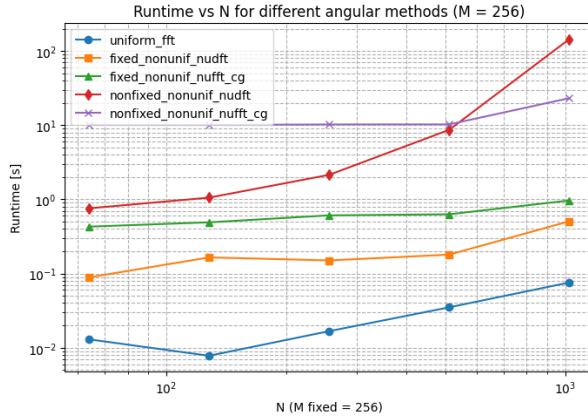


(a) Runtime vs. M (fixed N).

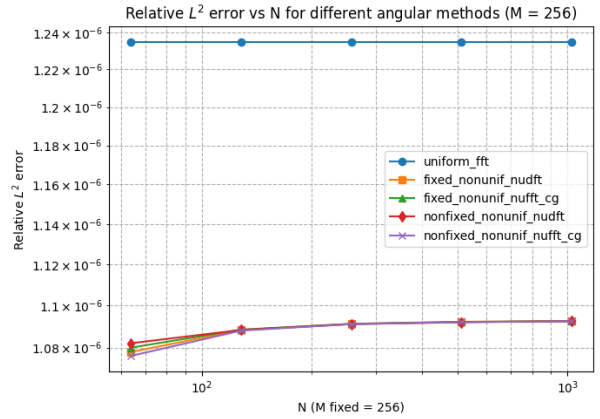


(b) Relative L^2 error vs. M (fixed N).

Figure 2: Timing and accuracy for fixed N and varying M .



(a) Runtime vs. N (fixed M).



(b) Relative L^2 error vs. N (fixed M).

Figure 3: Timing and accuracy for fixed M and varying N .

Table 5: Fixed N (here $N = 256$): runtimes [s] vs. M .

M	FFT	Fix-NU NUFFT	Fix-NU NUFFT	NF-NU NUFFT	NF-NU NUFFT
64	4.075e-03	1.875e-01	2.568e-01	6.018e-01	2.762e+00
128	7.578e-03	7.299e-02	3.131e-01	1.185e+00	5.839e+00
256	1.800e-02	1.791e-01	5.018e-01	2.337e+00	1.097e+01
512	3.710e-02	2.740e-01	1.072e+00	4.399e+00	2.030e+01
1024	8.701e-02	2.230e-01	1.976e+00	8.747e+00	4.870e+01

Table 6: Fixed M (here $M = 256$): relative errors vs. N .

N	FFT		Fix-NU NUFFT		Fix-NU NUFFT		NF-NU NUFFT		NF-NU NUFFT	
	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2	L_{rel}^{∞}	L_{rel}^2
64	7.285e-06	1.234e-06	7.300e-06	1.078e-06	7.307e-06	1.080e-06	7.298e-06	1.082e-06	7.305e-06	1.076e-06
128	7.285e-06	1.234e-06	7.289e-06	1.088e-06	7.288e-06	1.089e-06	7.290e-06	1.088e-06	7.288e-06	1.088e-06
256	7.285e-06	1.234e-06	7.285e-06	1.091e-06	7.286e-06	1.091e-06	7.286e-06	1.091e-06	7.286e-06	1.091e-06
512	7.285e-06	1.234e-06	7.285e-06	1.092e-06	7.285e-06	1.092e-06	7.285e-06	1.092e-06	7.285e-06	1.092e-06
1024	7.285e-06	1.234e-06	7.285e-06	1.093e-06	7.285e-06	1.093e-06	7.285e-06	1.093e-06	7.285e-06	1.092e-06

Table 7: Fixed M (here $M = 256$): runtimes [s] vs. N .

N	FFT	Fix-NU NUFFT	Fix-NU NUFFT	NF-NU NUFFT	NF-NU NUFFT
64	1.297e-02	8.877e-02	4.290e-01	7.577e-01	1.010e+01
128	7.797e-03	1.652e-01	4.906e-01	1.056e+00	1.009e+01
256	1.673e-02	1.504e-01	6.093e-01	2.142e+00	1.024e+01
512	3.506e-02	1.800e-01	6.297e-01	8.668e+00	1.027e+01
1024	7.575e-02	5.068e-01	9.671e-01	1.448e+02	2.315e+01

Fixed M

M vs N dominate show two problems where n vs m matters. probably a highly oscillatory one

8.3 Different Problems, Meshes, Conditions, etc...

Integration Schemes

Radial Meshes uniform vs random vs etc...

Azimuthal Meshes uniform vs random vs jittered etc...

Different Problems, Discontinuities, etc...

8.4 JCP Paper Problems

it appears the formulas from the original paper print out absolute values, not relative values. this matches both the matlab and my values. I will include the actual relative values

need to include mention of phi values etc... in neumann computation

Problem 1

$$u(x, y) = 3e^{x+y}(x - x^2)(y - y^2) + 5$$

$$f(x, y) = 6e^{x+y}xy(-3 + x + y + xy)$$

neumann values? or just leave out

Unit disk (r=1) table 1, relative errors (infimum) for N vs M, dirichlet table 2, fix N=64, infimum and 12 errors, varying M, trapezoidal and simpsons rule

this example is complete and in the JCPPaperproblem1 file in the github. it was run for uniform algorithm, NUFFT algorithm, and NUFFT algorithm. used a completely random nonuniform mesh

Problem 2

$$u(x, y) = (x+1)^{5/2}(y+1)^{5/2} - (x+1)(y+1)^{5/2} - (x+1)^{5/2}(y+1) + (x+1)(y+1)$$

$$f(x, y) = \frac{15}{4}(-((1+x)\sqrt{1+y}) + (1+x)^{5/2}\sqrt{1+y} - \sqrt{1+x}(1+y) + \sqrt{1+x}(1+y)^{5/2})$$

Unit Disk table 3, relative errors infimum and l2, fix N=64, vary M, dirichlet and neumann problems trapezoidal and simpsons rule

Problem 3

$$u(x, y) = \frac{e^x + e^y}{1 + xy}$$

$$f(x, y) = \frac{e^x (1 - 2y - 2x(-1 + y)y + 2y^2 + x^2(2 + y^2)) + e^y (1 + 2x(-1 + y) + 2y^2 + x^2(2 - 2y + y^2))}{(1 + xy)^3}$$

Table 4, n=64 varying M values, trapezoidal and simpsons, dirichlet and neumann, unit disk

Table 5, n=64 varying M values, trapezoidal and simpsons, dirichlet and neumann, B(0, .5)

Problem 4

$$u(x, y) = x^3 e^x (y+1) \cos(x+y^3)$$

$$f(x, y) = e^x x \left((1+y)(6+6x-9x^2y^4) \cos(x+y^3) - 2x(3(1+y) + x(1+y) + 3xy^2 + 3xy(1+y)) \sin(x+y^3) \right)$$

Table VI, relative errors for the Dirichlet and Neumann problems in the disk B(0; 1), n=64, varying m, trapezoidal and simpsons

The same problem was solved in the larger disk B(0; 2), and the numerical results are shown in Table VII. n=64, varying m, trapezoidal and simpsons

Problem 5

$$u(x, y) = \sin(\alpha\pi(x+y))$$

$$f(x, y) = -2\alpha^2\pi^2 \sin(\alpha\pi(x+y))$$

Table VIII, alpha = 5, M vs N, infimum error

Table IX, alpha = 20, M vs N, Infimum error

Problem 6

$$u(x, y) = 10\phi(x)\phi(y)$$

$$\phi(x) = e^{-100(x-1/2)^2} (x^2 - x)$$

$$f(x, y) = 20e^{-25(2x-1)^2} e^{-25(2y-1)^2} \left[(-10000x^4 + 20000x^3 - 14698x^2 + 4698x - 149)(y^2 - y) + (x^2 - x)(-10000y^4 + 20000y^3 - 14698y^2 + 4698y - 149) \right]$$

figure, analytical solution figure, solution errors. n=64, m=256, dirichlet, trapezoidal more errors, etc...

Problem 7

Problem 7. The last problem presents discontinuities on the boundary conditions.

The formulation is best described in polar coordinates:

$$\Delta u = f, \quad \text{in } B = B(0, 1),$$

$$u = g, \quad \text{on } \partial B,$$

where

$$f(re^{i\alpha}) = -4r^3 (\cos^2 \alpha \sin \alpha + \sin^3 \alpha) \sin(1 - r^2) - 8r \sin \alpha \cos(1 - r^2),$$

and

$$g(e^{i\alpha}) = \begin{cases} 0, & \alpha \in (0, \pi), \\ 1, & \alpha \in (\pi, 2\pi), \\ \frac{1}{2}, & \alpha \in \{\pi, 2\pi\}. \end{cases}$$

In this case we have the solution u given by

$$u(re^{i\alpha}) = \frac{1}{2} + \sin(r(1 - r^2) \sin \alpha) - \frac{2}{\pi} \sum_{k=1}^{\infty} \frac{r^{2k-1} \sin((2k-1)\alpha)}{2k-1}.$$

The actual input data is expressed in Cartesian coordinates as

$$f(x, y) = -4(x^2 y + y^3) \sin(1 - x^2 - y^2) - 8y \cos(1 - x^2 - y^2).$$

table 10, errors infimum N vs M. only take:

Note. Errors were taken only over the points in $B(0, 1) \setminus (B_{0.01}(1, 0) \cup B_{0.01}(-1, 0))$.

8.5 Comparison to other solvers

Other Methods

Accuracy

Timing

9 Conclusion

Summary

Future Directions Mention GPU acceleration using torch, cunumpy, or cuFINUFFT For the testing, we may want to use tamu supercomputer

10 Possible Reference Papers and Notes

- ON DIRECT METHODS FOR SOLVING POISSON'S EQUATIONS*B. L. BUZBEE, G. H. GOLUB: AND C. W. NIELSON:
very old paper from 1970. describes direct and efficient methods for poisson on the rectangle
- THE DIRECT SOLUTION OF THE DISCRETE POISSON EQUATIONON A DISK*PAUL N. SWARZTRAUBER" AND ROLAND A. SWEET:
Might be the earliest solver known. 1973 paper. describes the direct solution for poisson on disk with either dirichlet or neumann, although neumann in least squares sense. I believe they use finite differences.
- FINITE DIFFERENCE METHODS FOR POISSON EQUATION, LONG CHEN: This isnt a journal paper, seems more like class notes. but it has a basic introduction for finite difference methods in solving poissons equations.

References

- [1] Lori Badea and Prabir Daripa. A fast algorithm for two-dimensional elliptic problems. *Numerical Algorithms*, 30:199–239, 08 2002.
- [2] Eric Bergshoeff, Giacomo Giorgi, and Luca Romano. From relativistic gravity to the poisson equation. *Journal of High Energy Physics*, 2025(2):15, 2025.
- [3] Bernard Bialecki and Lyndsey Wright. A fast direct solver for a fourth order finite difference scheme for poisson’s equation on the unit disc in polar coordinates. *Numerical Algorithms*, 70, 05 2015.
- [4] Leonardo Borges and Prabir Daripa. A fast parallel algorithm for the poisson equation on a disk. *Journal of Computational Physics*, 169, 02 2001.
- [5] Steven Britt, Semyon Tsynkov, and Eli Turkel. A compact fourth order scheme for the helmholtz equation in polar coordinates. *J. Sci. Comput.*, 45:26–47, 10 2010.
- [6] Timothy R. Brumleve and Richard P. Buck. Numerical solution of the nernst-planck and poisson equation system with applications to membrane electrochemistry and solid state physics. *Journal of Electroanalytical Chemistry and Interfacial Electrochemistry*, 90(1):1–31, 1978.
- [7] Prabir Daripa and Ranjan Dash. A numerical study of pulsatile blood flow in an eccentric catheterized artery using a fast algorithm. *Journal of Engineering Mathematics*, 42, 07 2001.
- [8] Harry W. Davison. A digital computer program for calculating steady temperature and density distributions in a gas containing heat sources. NASA Technical Note NASA-TN-D-4135, National Aeronautics and Space Administration (NASA), Lewis Research Center, Cleveland, Ohio, September 1967.
- [9] Bengt Fornberg and David Merrill. Comparison of finite difference- and pseudospectral methods for convective flow over a sphere. *Geophysical Research Letters*, 24(24):3245–3248, 1997.
- [10] L. Gorelick, M. Galun, E. Sharon, R. Basri, and A. Brandt. Shape representation and classification using the poisson equation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(12):1991–2005, 2006.
- [11] Philip M. Gresho and Robert L. Sani. On pressure boundary conditions for the incompressible navier–stokes equations. *International Journal for Numerical Methods in Fluids*, 7(10):1111–1145, 1987.
- [12] Alina Jóźwikowska. Numerical solution of the nonlinear poisson equation for semiconductor devices by application of a diffusion-equation finite difference scheme. *Journal of Applied Physics*, 104(6):063715, 2008.
- [13] Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. Poisson surface reconstruction. In *Eurographics Symposium on Geometry Processing*, pages 61–70, Cagliari, Italy, 2006.
- [14] A. Klös and A. Kostka. A new analytical method of solving 2d poisson’s equation in mos devices applied to threshold voltage and subthreshold modeling. *Solid-State Electronics*, 39(12):1761–1775, 1996.
- [15] Ming-Chih Lai. A simple compact fourth-order poisson solver on polar geometry. *Journal of Computational Physics*, 182:337–345, 10 2002.
- [16] Ming-Chih Lai and Wei-Cheng Wang. Fast direct solvers for poisson equation on 2d polar and spherical geometries. *Numerical Methods for Partial Differential Equations*, 18(1):56–68, 2002.
- [17] Hongchun Mei, Fei Wang, Zhen Zeng, et al. A global spectral element model for poisson equations and advective flow over a sphere. *Advances in Atmospheric Sciences*, 33(4):377–390, 2016.
- [18] N. Mohammad and A. Trounev. An advanced algorithm for solving incompressible fluid dynamics: From navier–stokes to poisson equations. *The European Physical Journal Special Topics*, 234:2191–2208, 2025.

- [19] Eric Poisson and Clifford M. Will. *Gravity: Newtonian, Post-Newtonian, Relativistic*. Cambridge University Press, Cambridge, 2014.
- [20] R. Suero, M. A. V. Pinto, C. H. Marchi, L. K. Araki, and A. C. Alves. Analysis of algebraic multigrid parameters for two-dimensional steady-state heat diffusion equations. *Applied Mathematical Modelling*, 36(7):2996–3006, 2012.